

d-Matrix Raptor 3D-DRAM 生成式推理加速器（Hot Chips 2026）

来源：ServetheHome 现场直播报道 —— Hot Chips 2026，d-Matrix 演讲主题：*Raptor 3D-DRAM Accelerator for Generative Inference*

前序关联：d-Matrix 在 Hot Chips 2025 曾发布 Corsair SRAM 存内计算加速卡；同期推出 JetStream 400G 以太网 AI 推理加速网卡；本次是其 3D 堆叠 DRAM 路线的正式落地。

译者说明：以下章节完全对应原报道的 28 张 PPT（Slide 编号与原文保持一致，缺失 Slide 01/03/09/13/16 为原文未采用）；每张图已本地化保存至 images/ 目录，可离线浏览。

0. 演讲总览

d-Matrix 本次发布的核心论点可以用一句话概括：**SRAM 太小太烫、HBM 太慢太贵，生成式 LLM 推理（尤其 Decode 阶段）需要的“海量带宽 + 适中容量 + 可控功耗”恰好落在两者之间的空白区域——这正是 3D-DRAM 堆叠（逻辑 Die 堆叠在 DRAM Die 之上）的甜点区。**

Raptor 是这条路线的第一款量产级芯片：**TSMC N4 逻辑 Die + 3D DRAM Die，36 μm 面对面（Face-to-Face）键合**，宣称相比 HBM4 与 NVIDIA Rubin R200，单位面积带宽高 20 倍、单位带宽功耗低 13.5 倍。

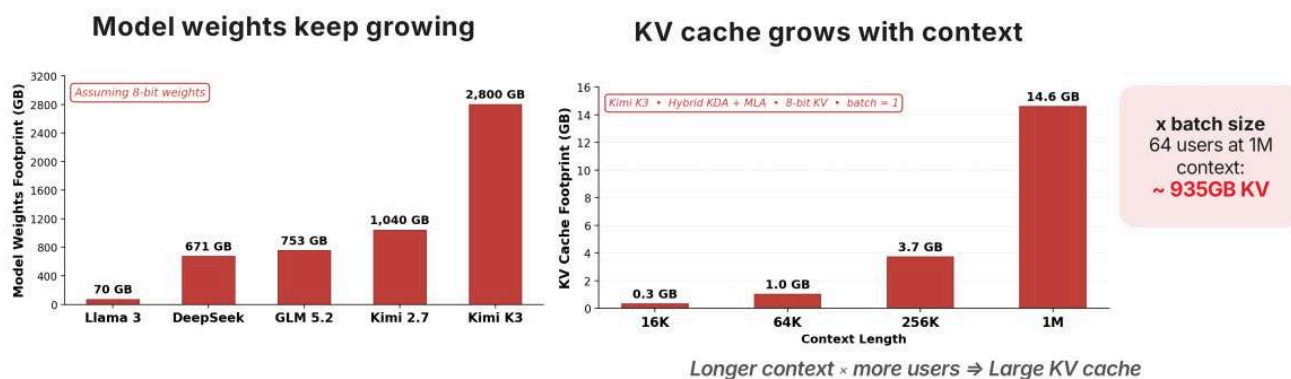
1. 问题背景 — LLM 推理的“双重数据爆炸”困境

原文定位：Paragraph 2（开头段二）+ Slide 02

随着大模型规模持续扩大，LLM 推理面临**两个独立维度同时增长的数据压力**：

- 模型权重（Weights）：**参数规模从 7B $\rightarrow$ 70B $\rightarrow$ 3T，容量压力持续存在；
- KV Cache（键值缓存）：**KV Cache 的容量 = 上下文长度 $\times$ Batch Size $\times$ 层数 $\times$ 通道数。当单卡 64 用户、1M 上下文时，KV Cache 总量约 **935 GB**，已经超过绝大多数单节点的显存容量。

The Growing Data Problem



Weights and KV cache present a **capacity and bandwidth problem**. Both are growing.

Slide 02: The Growing Data Problem — 模型权重与 KV Cache 同时膨胀，带来容量+带宽的双重瓶颈。

🔍 工程视角解读（译者补充）：

对 HBM 路线而言，容量通常通过多颗 HBM 堆叠来满足（如 NVIDIA GB200 配 8~16 颗 HBM3e），但带宽的提升依赖两个瓶颈：引脚速率（pin speed）和每颗 Base Die 上的 I/O 通道宽度——两者都在逼近物理极限。这正是 d-Matrix 切入的论证起点。

2. 现有两种存储架构的困境（非此即彼的两难）

2.1 SRAM 路线：带宽无敌，但容量&功耗不可接受

原文定位：Paragraph 3 + Slide 04

SRAM（例如 d-Matrix 上一代 Corsair 加速卡）的优势是**极致带宽**：一对 Corsair SRAM 加速卡可实现 ~300 TB/s 的读带宽、约 1 ns 延迟。但它的硬伤同样致命：

- **容量极小**：一对卡总共仅能装约 **4 GB**；
- **面积成本高**：一个 6T SRAM Cell（6 晶体管静态存储单元）的面积约是 DRAM Cell 的 **10 倍**；
- **漏电功耗爆炸**：GB 级容量的 SRAM 阵列漏电即达数十瓦量级。

SRAM: Bandwidth Advantage

Why SRAM is fast:

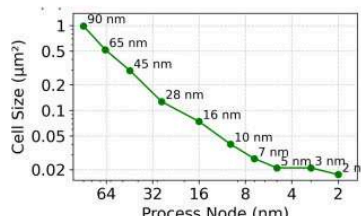
- 6T cell, direct bit-line access
- On-die: no interposer, no package crossing
- Sub-ns access latency

Corsair SRAM Accelerator Specs – card pair:

Metric	Value
Bandwidth	300 TB/s
Capacity	4 GB
Latency	~1 ns
I/O energy	~0.5 pJ/bit

Why SRAM cannot scale:

- 6T cell is **10× larger** than DRAM 1T1C
- Bitcell frozen ~**0.021 μm^2** across N5 → N3E → N2 (TSMC IEDM/ISSCC)
- Leakage: tens of watts at GB scale
- Practical limit: ~**4 GB per card pair**
- Cost: **100x** DRAM



Source: [Dayo et al.](#)

SRAM is the fastest memory in silicon.
Example use case: Draft model in frontier-LLM speculative decoding.

Slide 04: SRAM Bandwidth Advantage — 带宽极高，但容量仅 GB 级，且漏电代价沉重。

d-Matrix 对此给出了一个场景化结论：**SRAM 更适合“推测解码（Speculative Decoding）”里的 Draft 模型（小模型快速产生候选 token，再用大模型验证），而不适合装前沿大模型的全量权重。**（原文提到 NVIDIA 正在以 Groq 的小模型为 Draft 做推测解码，即此例。）

2.2 HBM 路线：容量可接受，但带宽天花板+功耗天花板同时到来

原文定位：Paragraph 4-5 + **Slide 05、Slide 06**

HBM（High Bandwidth Memory，高带宽显存）解决了容量问题，但它在两个方面撞上了物理墙：

(1) 带宽增长速度停滞（Slide 05）

HBM 的总带宽取决于：

- 每颗 Base Die 的 pin 速率（提升慢）
- 每颗 Base Die 的 I/O 宽度（每代提升有限）
- 单封装内堆叠颗数（受封装边缘可用面积 beachfront 限制，普遍 8~16 颗封顶）

按 d-Matrix 估算，即使 HBM4 时代，NVIDIA Vera Rubin / AMD Instinct MI455 这类顶级封装的实际可用带宽天花板约 **20 TB/s**。

HBM: The Bandwidth Issue

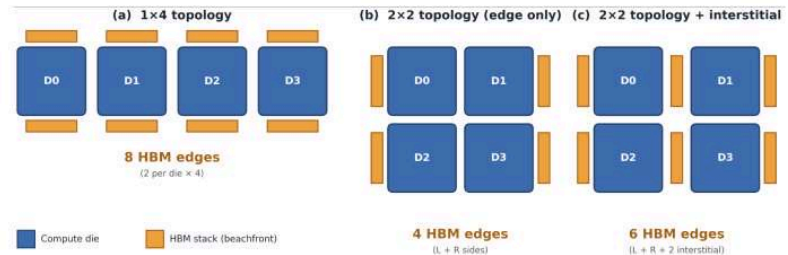
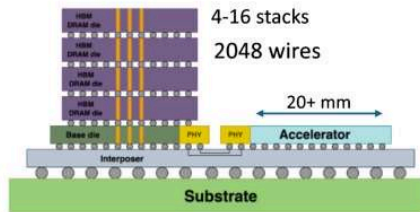
HBM provides high capacity

- 1T1C cell, high array density
- 8–16 core dies per stack
- 8 stacks per package

HBM BW is challenging to scale to SRAM-like numbers

- Pin speed, IO width (per HBM base die) have improved slowly
- Number of stacks is beachfront limited
- Dedicating the entire die beachfront to HBM forces harder packaging (interposer size, warpage). Examples of 4 die package beachfronts
- A Practical limit w/ HBM4 (Vera Rubin, MI 455): ~20TB/s

HBM-4

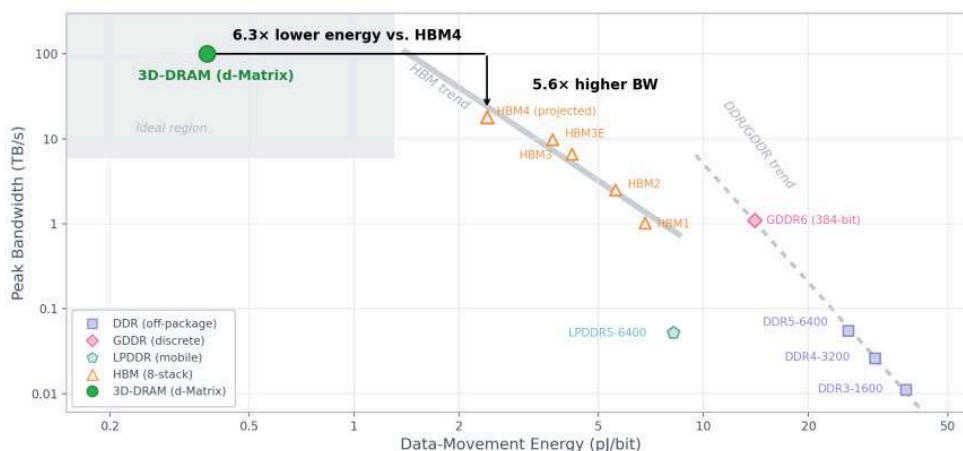


Slide 05: HBM The Bandwidth Issue — pin 速率、I/O 宽度、封装可用边缘面积三重约束使 HBM 带宽增长停滞。

(2) 功耗无法承受 (Slide 06)

按 HBM4 典型能效 **2.4 pJ/bit** 计算，推动 **100 TB/s** 带宽仅数据搬运就需要 **1.92 kW**，还未计入片上 NoC/互连 Fabric 的开销。而当前封装既没有足够 beachfront 面积去堆 HBM 到 100 TB/s，也没有相应的热与供电预算。

HBM: The Power Problem



At 2.4pJ/bit, 100TB/s requires 1.92KW for HBM power only (excluding fabric power)

HBM capacity is very high but SOTA packages neither have beachfront, nor power for SRAM-like bandwidth.

3. d-Matrix 的核心答案：3D-DRAM 堆叠（逻辑 Die 在 DRAM Die 之上）

3.1 技术主张：为什么是现在？

原文定位：Paragraph 6 + Slide 07

d-Matrix 的方案非常直接：直接把计算（逻辑 Die）堆叠在 DRAM Die 上方，而不是像 HBM 那样把 DRAM 堆叠在逻辑 Base Die 上方，再通过 Interposer 走 cm 级互连。

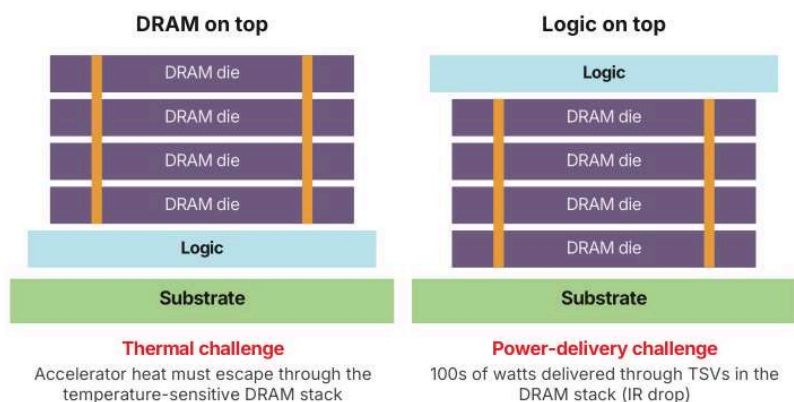
这当然会带来两个经典挑战：

- **散热挑战**：几百瓦的热必须从上层的热逻辑 Die 通过 TSV 垂直散出，而 DRAM 本身对温度非常敏感（ $> 95^{\circ}\text{C}$ 后数据保持时间急剧下降）；
- **供电挑战（IR Drop）**：高密度垂直供电网络会带来压降，必须保证逻辑 Die 与 DRAM Die 都在规格内工作。

d-Matrix 给出了可量产的可行性边界：

1-Hi 堆叠（逻辑在上、DRAM 在下），逻辑 Die 面功率密度 $\leq 0.5 \text{ W/mm}^2$ ，配合液冷即可保证 DRAM Die 温度 $\leq 100^{\circ}\text{C}$ 。

3D-DRAM: An idea whose time has come



1-Hi stack, Logic-on-Top with power density $\leq 0.5 \text{ W/mm}^2$ can be reliably liquid-cooled (DRAM $< 100^{\circ}\text{C}$) with required power delivery.

Slide 07: 3D-DRAM An idea whose time has come — 在 $0.5\text{W}/\text{mm}^2$ 功率密度 + 液冷条件下，1-Hi 逻辑在上的堆叠可把 DRAM 温度控制在 $\leq 100^\circ\text{C}$ 。

3.2 能效阶梯：3D-DRAM 位于 SRAM 与 HBM 之间的"能量甜点区"

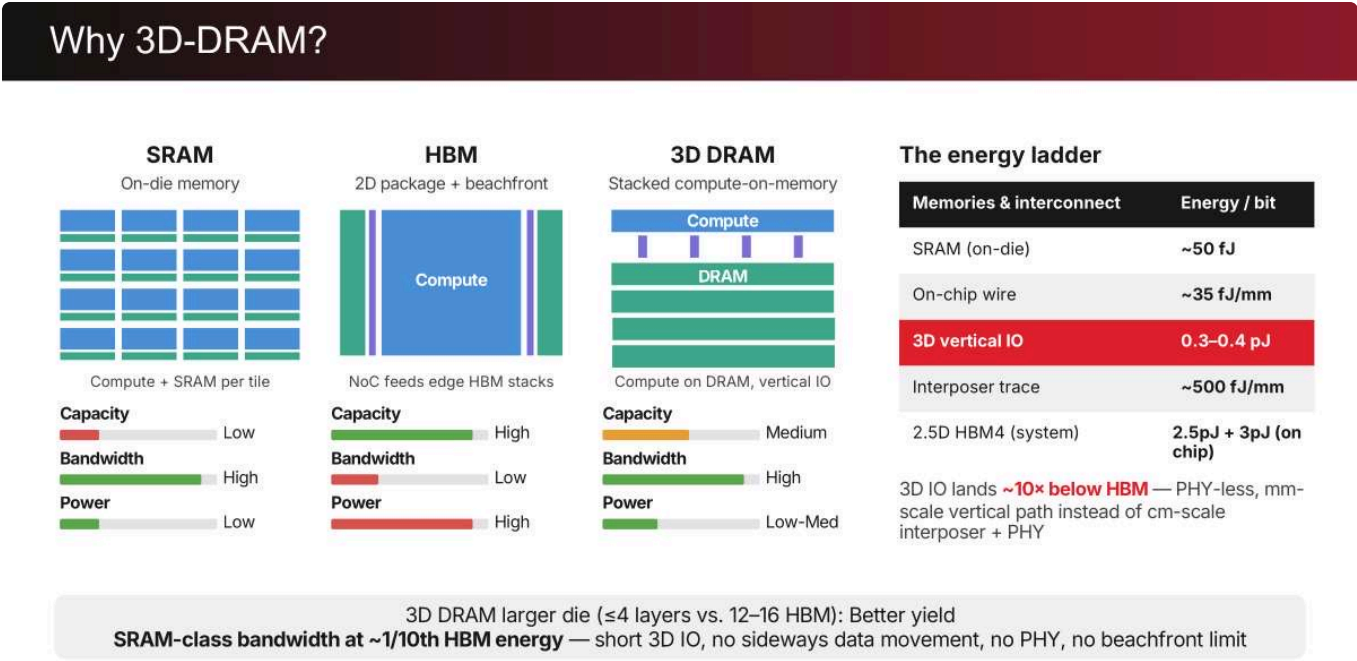
原文定位：Paragraph 7 + Slide 08

d-Matrix 给出了一条从"最近的存储"到"最远的存储"的能量阶梯（单位 bit 搬运能耗）：

技术形态	典型搬运能耗	备注
片内 SRAM（On-die SRAM）	~50 fJ/bit	最近、最省，面积 $10\times$ DRAM
3D-DRAM 垂直 I/O （Raptor 方案）	~0.3~0.4 pJ/bit	mm 级路径，无 PHY，比 HBM 低 $10\times$
HBM4（2.5D Interposer 路径）	~2.5~5 pJ/bit	cm 级硅中介层路径，需 PHY

核心原因非常朴素：3D 堆叠的垂直 I/O 路径只有 mm 级，不需要复杂的 SerDes/PHY（物理层接口电路），直接走毫米级面内互连线；而 HBM 的 Interposer 是 cm 级路径，且 PHY 功耗占了大头。

附带的额外好处：相比 HBM 动辄 8-Hi/12-Hi 堆叠，3D-DRAM 堆叠层数少（1-Hi），逻辑 Die 可以做得更大，良率反而更优。



Slide 08: Why 3D-DRAM? — 垂直 3D I/O ($\sim 0.3\text{ pJ/bit}$) 介于 SRAM ($\sim 50\text{ fJ}$) 与 HBM4 ($\sim 2.5\sim 5\text{ pJ}$) 之间，比 HBM 低 $10\times$ 。

4. 负载映射：3D-DRAM 恰好命中 LLM Inference 的瓶颈——Decode 阶段

4.1 LLM 推理两阶段的瓶颈差异：Compute-Bound vs Memory-Bound

原文定位：Paragraph 8 + Slide 10

不同 LLM 推理阶段、不同模型变种，瓶颈完全不同：

阶段/算子	典型计算模式	瓶颈类型	关键诉求
Prefill（预填充）	大量提示词 token 并行计算，GEMM 大矩阵乘	计算吞吐受限（Compute-Bound）	高算力密度
Decode（逐 token 生成）	一次生成 1 个 token，GEMV 向量×矩阵	内存带宽受限（Memory-Bound）★	超高带宽
Attention（高 GQA / 推测解码时）	GQA（分组查询注意力）+ 候选 token 重打分	可能翻转到 Compute-Bound	算力
MoE（混合专家模...	只激活少数专家	即使中等 batch 也保持 Memory-Bound	带宽

结论非常明确：Decode 阶段恰好是对“超高带宽”需求最强的阶段——而这正是 3D-DRAM 的主场。

LLM Inference

Prefill Stage

Prompt tokens (all, in parallel)

t1

t2

t3

t4

t5

Transformer layers × N

Multi-Head Attention

Feed-Forward (FFN)

write

KV cache

→ first token, then hand to Decode

Decode Stage

Current token (1)

t_k

Transformer layers × N

Multi-Head Attention

Feed-Forward (FFN)

append

KV cache

read

t_{k+1}

Next token

autoregressive (repeat per token)

Agentic

reuse prior KV (no recompute)

phases

Prefill T1

Decode T1

Prefill T2

Decode T2

KV cache

User 1 prompt

Assistant 1 generated

User 2 prompt

Assistant 2 generated

Turn 1

Turn 2 (appended)

- Prefill Stage is **compute-throughput bound**
 - Large M dimension from processing multiple tokens at a time (irrespective of batch size)
- Decode Stage is typically **memory-bandwidth bound**
 - Small M dimension (small, modest batch sizes for low-latency scenarios)
 - Attention can become compute bound with high GQA + speculative decoding (independent of batch size)
 - MoE is memory-bandwidth bound even for modest batch sizes + speculative decoding

Slide 10: LLM Inference — Prefill 算力受限、Decode 带宽受限、Attention/MoE 依情况切换；
Decode 最需要海量带宽。

4.2 Decode 阶段占据绝大多数墙钟时间

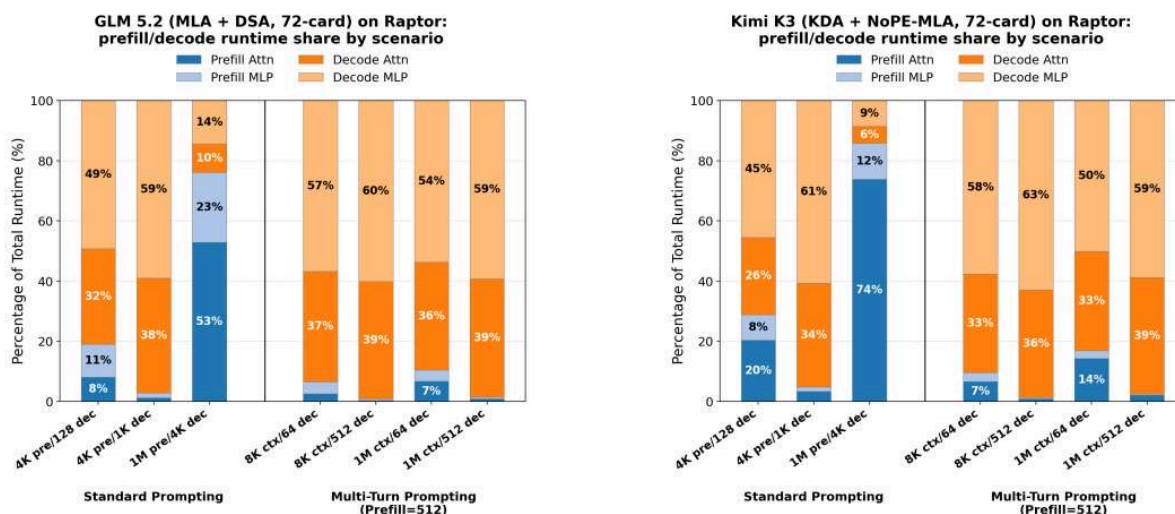
原文定位：Paragraph 9 + Slide 11

d-Matrix 给出的实证数据：端到端推理中，绝大多数墙钟时间（Wall-Clock Time）消耗在 Decode 阶段。因此，哪怕 Prefill 性能不占优，只要能把 Decode 带宽拉上去，整体 TPS（每秒 token 数）就会显著提升。

🔍 工程视角解读（译者补充）：

这与我们前面讨论 LPDDR5X-PIM / DCIM 时反复强调的"Decode 是 GEMV，权重复用率只有 1×，瓶颈 100% 在搬运"完全一致。d-Matrix 的架构选择实际上是押注一个判断：未来低延迟推理的系统优化焦点会越来越集中在 Decode 带宽，而不是 Prefill 算力密度。

Majority of wall-clock inference time is spent in decode



Decode Phase (memory bound component) is a critical part of overall inference runtime.

Slide 11: Majority of wall-clock inference time is spent in decode — 优化 Decode 带宽 = 优化整体端到端性能。

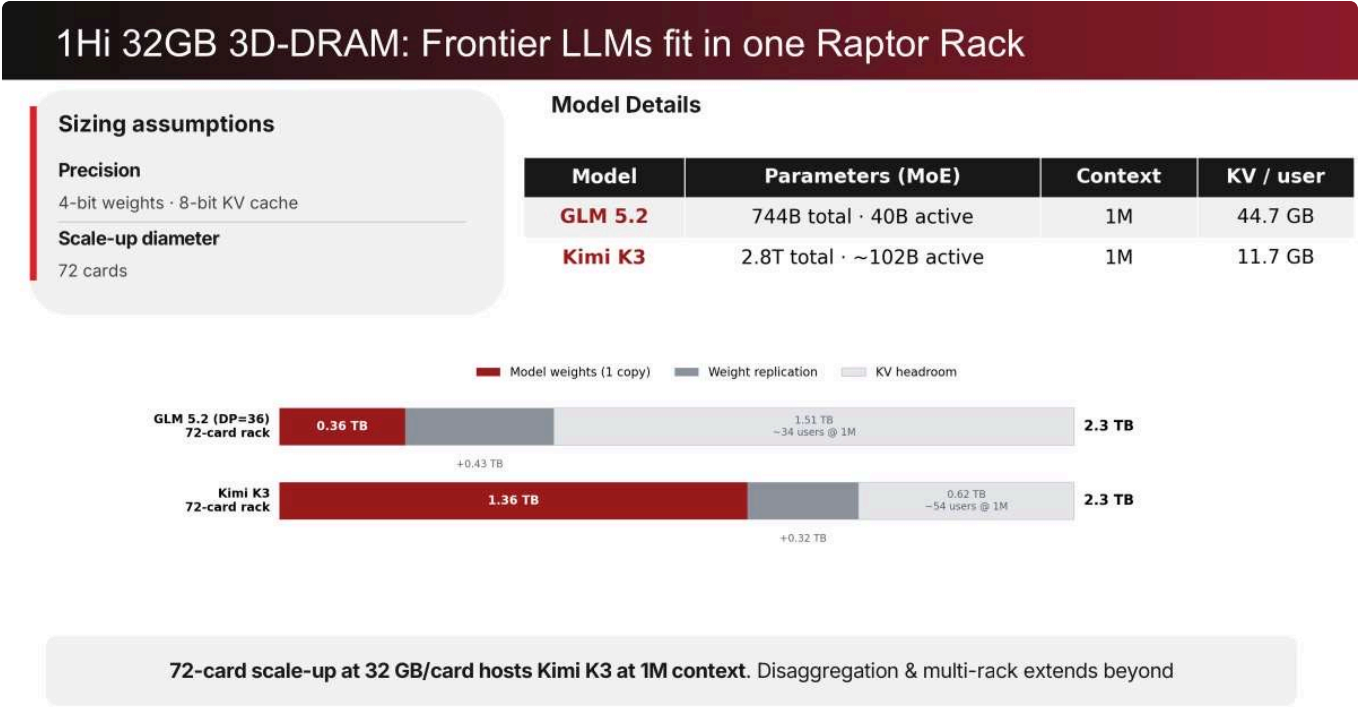
5. 部署规模：32GB 单卡 × 72 卡一机架 = 前沿 LLM + 1M 上下文装下

原文定位：Paragraph 10 + Slide 12

Raptor 单卡容量为 **32 GB 3D-DRAM**。在 4-bit 权重 + 8-bit KV Cache 的工程假设下：

- 单卡容量和带宽能保证低延迟推理；
- 扩展到 **72 卡的机架级（Rack Scale-up）** 即可完整部署 **Kimi K3 这类万亿参数级前沿模型 + 1M 上下文长度**；
- 如需更大规模，可通过解耦（Disaggregation）和跨机架互联继续扩展。

这正是 d-Matrix 的核心理念落地：“**适中的单卡容量（32GB，远比 SRAM 的 4GB 大）+ 单卡极高带宽（~100 TB/s 级，远比 HBM 的 20TB/s 天花板高）**”的组合，在 LLM Decode 推理场景恰好形成闭环。



Slide 12: Frontier LLMs fit in one Raptor Rack — 72 卡 × 32GB 即可装 Kimi K3 1M 上下文模型。

6. 系统级协同设计三角：存储、互连、负载映射缺一不可

原文定位：Paragraph 11 + Slide 14

Raptor 不是单靠一颗 3D-DRAM Die 就能赢的。d-Matrix 强调必须做三层协同设计（Co-Design Triangle）：

1. **存储子系统（Memory Subsystem）**：3D-DRAM 的 bank 映射、纠错、刷新、可靠性机制；
2. **数据移动互连（Data Movement Fabric）**：包内（Intra-Card）与跨卡（Inter-Card）的低延迟全 Mesh 网络；

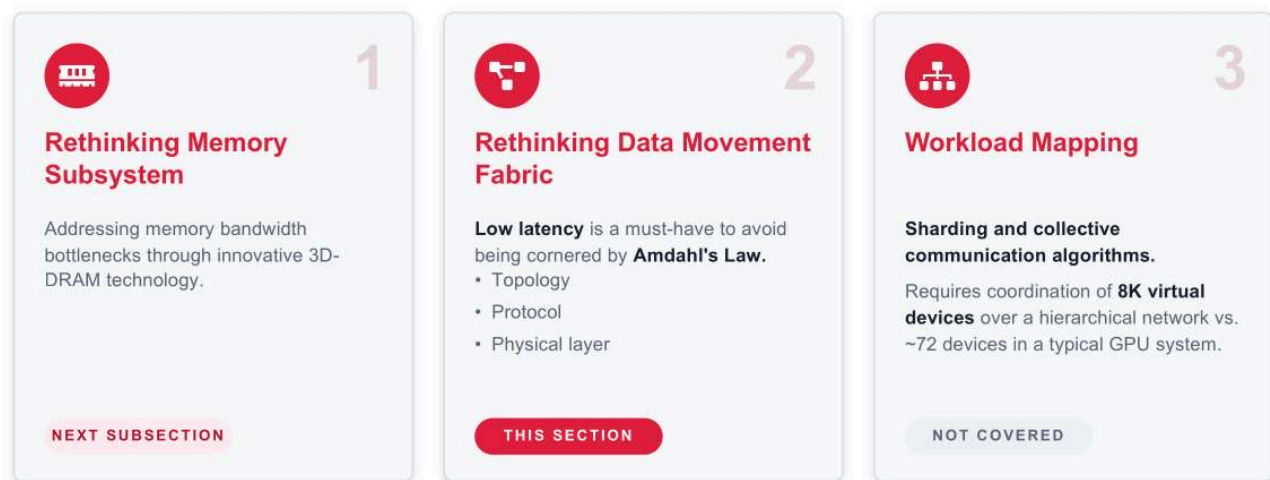
3. **负载映射 (Workload Mapping)**：如何把 Attention/MLP/MoE 各层映射到 3D-DRAM 的通道/bank/tile 上，最大化带宽利用率、最小化跨片搬运。

 呼应你之前的 LPDDR6-PIM 工作：

这三层三角与你正在设计的"LP6 bank 旁 PU 架构"高度同构——存储映射 (DataMapper + 2D Reshape)、互连/聚合网络、负载分块策略，三者正是任何 PIM/存算一体系统能否落地的三个"咬合齿轮"。

Building a 3D-DRAM Based Inference System

COMPLEX CO-DESIGN



Hot Chips 2026 | d-Matrix

Slide 14: Building a 3D-DRAM Based Inference System — 存储、互连、负载映射三者必须协同设计。

7. 低延迟互连 Fabric：全 Mesh 包内 + 跨卡拓扑

原文定位：Paragraph 12 + Slide 15

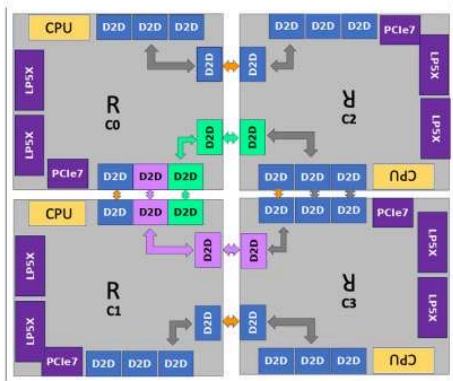
Raptor 芯片之间、卡之间使用 **低延迟全 Mesh (Full Mesh)** 的互连 Fabric：

- **Intra-Card (卡内多 Die 之间)**：通过封装内的全 Mesh 路由实现 tile 间低延迟直接通信，避免大量数据回到宿主 CPU/NPU 再转发。
- **Inter-Card (跨卡之间)**：配合 d-Matrix 已推出的 JetStream 400G 以太网网卡，在机架/数据中心尺度组成 AI 推理集群 Fabric。

这正是 Decode 场景的必须要素：GEMV 阶段需要从大量 3D-DRAM tile 并行读取权重，低延迟 Fabric 才能把结果聚合、写回而不成为瓶颈。

Low Latency Fabric (Intra-Card and Inter-Card)

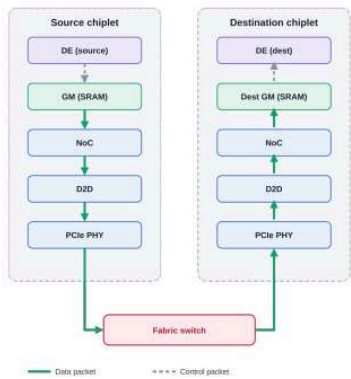
TOPOLOGY



- **Full-mesh package** reduces intra-package latency.
 - Realized via "virtual" diagonals (green / magenta D2D).
 - Physical diagonals increase distance (package size, D2D power) and package layers.

Hot Chips 2026 | d-Matrix

PROTOCOL



- **Push-style** end-to-end — intra-card and inter-card.
 - Device initiated.
 - Single-sided — source card doesn't receive an ACK.
 - Efficient support for multiple in-flight short-message transfers on a card.

Slide 15: Low Latency Fabric (Intra-Card and Inter-Card) — 全 Mesh 拓扑保证聚合与广播低延迟。

8. Raptor 芯片实现：TSMC N4 逻辑 Die F2F 堆叠在 3D DRAM Die 上

原文定位：Paragraph 13 + Slide 17

芯片本身的实现参数（原文明确给出）：

项目	规格
堆叠方式	1-Hi，逻辑 Die 面朝下（F2F，Face-to-Face）堆叠在 DRAM Die 上
键合间距	36 μm （hybrid bonding 间距级别）
逻辑 Die 工艺	TSMC N4（4nm 级）
DRAM Die	3D-DRAM（定制 DRAM 工艺，与逻辑 Die 垂直高密度互联）
工艺成熟度宣称	d-Matrix 称该 F2F 流程"已验证成熟、低成本、可高量产、良率高"

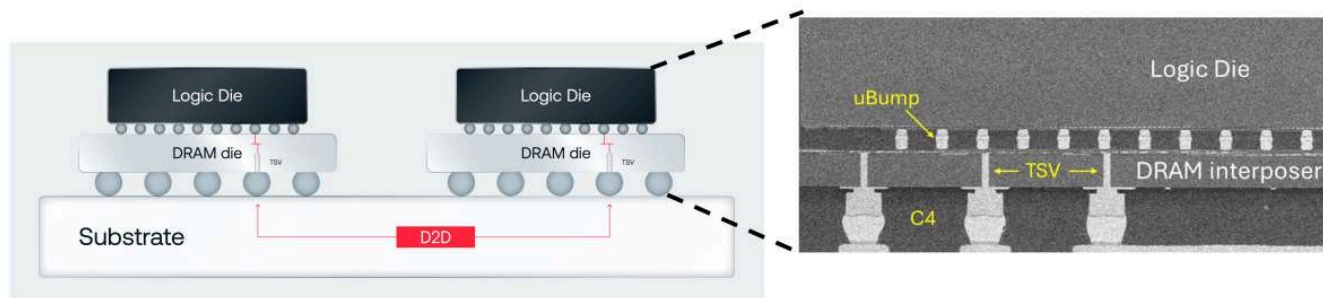
和三星 LPDDR5X-PIM 的差异对比（译者补充）：

三星 LPDDR5X-PIM 走的是"在 DRAM bank 旁做小规模 MAC tree + 寄存器"的 in-DRAM logic 路线；Raptor 走的是"逻辑 Die 在上、DRAM Die 在下通过垂直 I/O 直连"的 3D-stacked near-DRAM logic 路线。

相同点：都想把计算搬到"离 DRAM cell 最近的位置"，用 ~ 0.3 pJ/bit 级的短路径替代 HBM 的几 pJ/bit。

差异点：Raptor 的逻辑 Die 是 N4 高性能工艺，可放复杂张量引擎/NoC；而 LPDDR5X-PIM 的逻辑只能做在 DRAM 工艺上，晶体管密度与性能受限。

Raptor 3D-DRAM



- Top die: TSMC N4 logic
- Bottom die: 3D DRAM
- Integration: 36μ Face to Face (F2F) stacking
- Proven low cost, high volume, high yield process

	HBM4	3D-DRAM
Topology	2.5D (beside)	F2F (below)
I/O energy	2-3 pJ/bit	0.37 pJ/bit
BW / card	18 TB/s	100+ TB/s
Capacity	192 GB	32 GB
5.6x higher BW / card 5-8x lower energy / bit 7x denser I/O		

Slide 17: Raptor 3D-DRAM — TSMC N4 逻辑 Die 36μ F2F 键合于 3D DRAM Die 之上。

9. 集成难题全景 & 三大挑战的相互耦合

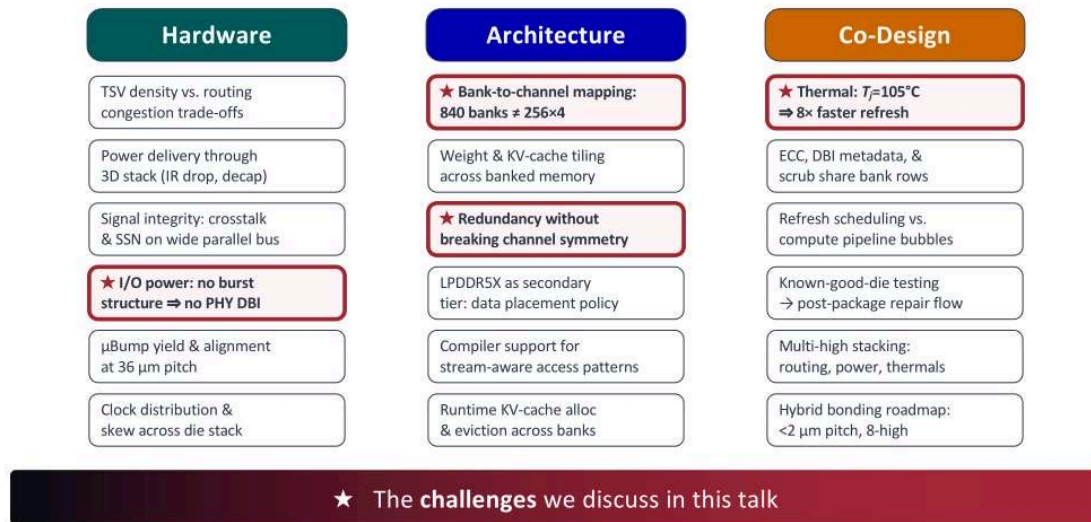
9.1 集成全景挑战（Four Challenges Landscape）

原文定位：Paragraph 14-15 + Slide 18

d-Matrix 把 3D-DRAM 加速器的集成挑战归纳为四大问题，后续重点展开其中三个：

- **Bank-to-Channel 映射问题**（张量引擎需要的 128B flit 如何从 32B/bank 的 DRAM 读出中高效拼装）；
- **I/O 功耗墙**（100 TB/s 级带宽下，仅 I/O 翻转就几百瓦）；
- **热可靠性**（ 105°C 结温下 DRAM 的保持时间、良率、软错误、刷新风暴）；
- 以及封装/测试等其它集成问题。

The 3D-DRAM Integration Landscape



Slide 18: The 3D-DRAM Integration Landscape — 四大类集成挑战需系统性解决。

9.2 三大挑战高度耦合（Entangled）

原文定位：Paragraph 15 + Slide 19

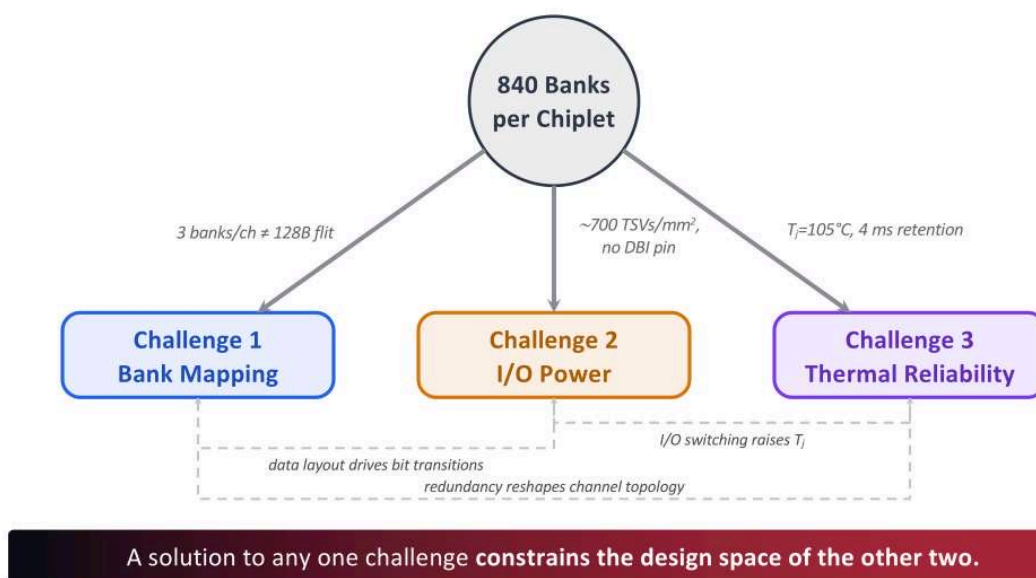
这是 d-Matrix 演讲中最“硬核工程”的一页：三个问题不是独立的，任何一个问题的解决方案都会压缩另外两个的设计空间。

举例：

- 如果你为了解决 Bank-to-Channel 映射去做“列交错+移位缓冲”，会增加 I/O 路径的切换，恶化 I/O 功耗；
- 如果你为了 I/O 功耗去做额外反相/DBI 电路，会增加热密度，恶化 DRAM 可靠性；
- 如果你为了可靠性降低刷新间隔（更多刷新），又会反过来吞掉 Bank 可用带宽，映射问题更严重。

这意味着：Raptor 的三个解决方案（Stream Blocking、Stream Flipping、Thermal-Aware ECC/Refresh + Bank Chaining）必须联合设计，而不是各做各的。

These Challenges Are Entangled



Slide 19: These Challenges Are Entangled — 任一问题的解法都会约束另外两个的设计空间。

10. 挑战 1：Bank-to-Channel 映射与 Overfetch（带宽浪费 33%）问题

10.1 问题根源：3 banks/channel × 32B = 96B，凑不齐张量引擎的 128B flit

原文定位：Paragraph 16-17 + Slide 20、Slide 21

前提 1（计算侧需求）：Raptor 每个张量引擎（Tensor Engine）每次计算需要一个 **128 Byte** 的 **flit**（数据包/访问粒度）。

前提 2（存储侧供给）：DRAM 每次 column 访问（列访问）从一个 bank 返回 **32 Byte**，因此理论上每 channel 需要 4 个 bank 并行（4×32=128B）。

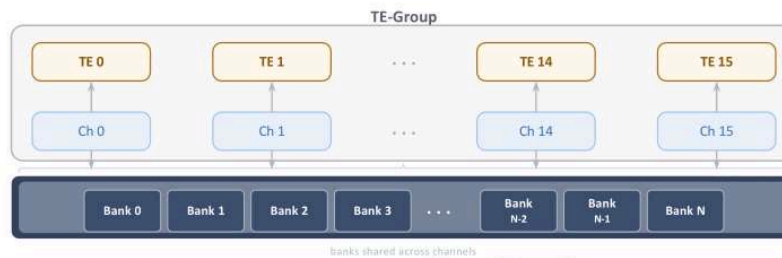
矛盾出现了（Slide 20）：

Raptor 这颗 3D DRAM Die 总共 840 个 banks（其中 72 个做冗余 spare，实际可用 768 个），分布在 **256 个 channel** 上 —— 即 **每 channel 只有 3 个 banks**：

$$\left[\frac{768 \text{ banks}}{256 \text{ channels}} = 3 \text{ banks/channel} \right]$$

3 个 banks 一次并行只读 3×32B = 96B，凑不齐计算所需的 128B flit。

Challenge 1: The Bank-to-Channel Mapping Problem



The setup

- Each TE needs a **128 B flit** per access
- $16 \text{ ch} \times 16 \text{ TE-Groups} = \mathbf{256 \text{ ch/chiplet}}$
- Each bank delivers **32 B** per column access

The arithmetic

- $128 \text{ B} \div 32 \text{ B} = 4 \text{ banks/ch needed}$
- DRAM die: **840 banks** (768 after 72 spares)
- $768 \div 256 = \mathbf{3 \text{ banks/ch available}}$

Need 4 banks/channel $\Rightarrow$ have only 3 $\Rightarrow$ **the 128 B flit does not divide evenly**

Slide 20: Challenge 1: The Bank-to-Channel Mapping Problem — $768/256 = 3 \text{ banks/channel} < 4$, 读不到 128B。

直接后果：Overfetch Dilemma（过取两难）（Slide 21）

为了拿到 128B，必须做**两次 3-bank 访问**（共 $2 \times 96\text{B} = 192\text{B}$ ），多取的 64B 被直接丢弃，结果：

- 约 **33% 的带宽被白白浪费**（在 33 TB/s 的目标下，就意味着 $\sim 11 \text{ TB/s}$ 的读出带宽无用）；
- 若尝试用"列交错拼接（Column Staggering）+ 192B 移位缓冲"来零浪费拼装，则需要复杂的时序校正与移位网络，且大幅提升验证难度和面积成本。

Challenge 1: The Overfetch Dilemma



With 3 banks per channel:

- One channel access returns **96 B** ($3 \times 32 \text{ B}$)
- 128 B flit $\Rightarrow$ two accesses $\Rightarrow$ 192 B fetched
- **33% overfetch $\approx$ 33 TB/s wasted**

Naïve fix — column staggering:

- Stagger column indices to pack one flit per pair
- Requires a **192 B shifting buffer**
- Complicates timing closure and verification

How to deliver 128 B from $3 \times 32 \text{ B}$ banks **without** wasting bandwidth or adding complex datapaths?

Slide 21: Challenge 1: The Overfetch Dilemma — 两种朴素方案均不可接受。

10.2 解决方案：Stream Blocking（流式分块——跨 3 个 flit 共享一次部分访问）

原文定位：Paragraph 18 + Slide 22

d-Matrix 的解法叫 **Stream Blocking**，核心是“把若干个 128B flit 打包成一组，让每组内部的 32B 碎片互相拼接，而不是在单个 flit 内试图拼满”：

具体规则（Slide 22 图示）：

- **让每 3 个 flit 为一组**，组内共享“最后一个不完整的 32B 读出”；
- **做 4 次 96B 的 3-bank 访问**（输入总量 $4 \times 96 = 384 \text{ B}$ ），刚好喂给 3 个需要 128B 的 flit（输出总量 $3 \times 128 = 384 \text{ B}$ ）。

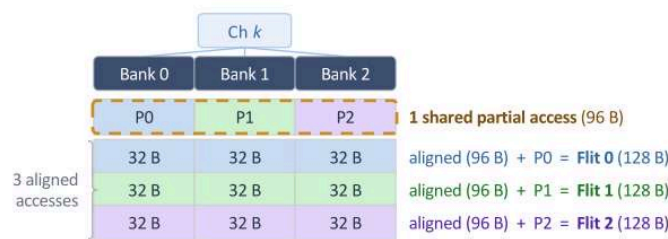
结果：

- ☒ **Overfetch = 0**，每次 column 访问的 32B 都被用上；
- ☒ **无需额外的 192B 移位网络**，时序/验证难度回落到正常水平。

这本质是在更大粒度上做“带宽碎片的拼接”，牺牲一点流式分组缓冲，换取了带宽零浪费与低复杂度。

Solution: Stream Blocking

Challenge. A 128 B flit = 4×32 B, but each channel has only **3 banks** — a naïve fetch reads 192 B per 128 B flit ($\Rightarrow$ 33% overfetch).



Puts the 4th 32 B (the *partial*) into one access *shared* by 3 flits — $4 \times 96 = 384$ B in = $3 \times 128 = 384$ B out $\Rightarrow$ 0% overfetch.

Every column access is fully utilised — the **33 TB/s** lost to overfetch is reclaimed, with **no shifting network**.

Slide 22: Solution: Stream Blocking — 跨 3 flit 共享 1 次部分 32B 访问，消除 33% 带宽浪费。

11. 挑战 2：I/O 功耗墙（100 TB/s 仅 I/O 就需 ~296 W）

11.1 问题：传统 DBI 方案需要额外 pin，而 3D 单周期链路没有"突发"概念

原文定位：Paragraph 19-20 + Slide 23

按 Raptor 目标的 **100 TB/s** 有效带宽，结合 3D I/O 的 **0.37 pJ/bit** 能效做乘法：

$$\left[100 \times 10^{12} \frac{\text{Byte}}{\text{s}} \times 8 \frac{\text{bit}}{\text{Byte}} \times 0.37 \times 10^{-12} \frac{\text{J}}{\text{bit}} \right] \approx 296 \text{ W}$$

仅 I/O 翻转一项就接近 **300 W**。

传统 DRAM/HBM 用来降 I/O 翻转功耗的成熟手段叫 **DBI (Data Bus Inversion, 数据总线反转)**：如果一帧数据里 1 比 0 多，就整体取反并加一个 DBI 标志位，平均节省约 20% 翻转。

但 3D 链路用不了传统 DBI (Slide 23)：

- HBM/LPDDR 是多周期突发 (Multi-cycle burst)，PHY 能看到完整 burst 再决定是否取反；

- 而 Raptor 的 3D 链路是**单周期（Single-cycle）256-bit 传输**——没有突发，且没有多余 pin 来走 sideband 标志位。

Challenge 2: The I/O Power Wall

The power cost of 100 TB/s:

Measured I/O Power

$$P_{I/O} = 100 \text{ TB/s} \times 0.37 \text{ pJ/bit} = \mathbf{296 \text{ W}}$$

Why conventional DBI cannot help:

- HBM uses multi-cycle bursts (BL 4–16)
- PHY sees full burst $\Rightarrow$ decides on inversion
- DBI pin per byte lane signals choice

Our 3D-DRAM has **none of these**: single-cycle transfer, no burst, no sideband pin.

Interface comparison:

DDR / HBM	
Transfer	Multi-cycle burst
Lookahead	Full burst visible
DBI	Dedicated pin/lane
Our 3D-DRAM	
Transfer	Single-cycle 256-bit
Lookahead	None
DBI	No pin. No sideband.

DBI would save 20% power

Need to rethink interface.

How to achieve DBI architecturally?

Slide 23: Challenge 2: The I/O Power Wall — 单周期 256-bit 链路无 burst 无 sideband pin，传统 DBI 无法实现。

11.2 解决方案：Stream Flipping (Pinless DBI —— 无需额外 pin 的 DBI)

原文定位：Paragraph 20 + Slide 24

Raptor 的思路是：**DBI 的本质是"减少翻转"**，不一定非要是"和 0/1 比较取反"；改为"和前一周期的数据比较取反"一样能减少翻转，而且只需要 1 bit metadata 存在已有 ECC 的旁位里。

具体机制 (Pinless DBI / Stream Flipping)：

- 发送端把本 flit 与**上一个 flit**按位比较；
- 若 Hamming 距离大（翻转太多），则本 flit **整体按位取反**再发送；
- 把"本次是否被翻转"这 1 bit metadata，**嵌在 ECC 校验位的旁边**（与纠错位共享存储空间），**不需要任何额外 pin**；
- 接收端根据该 metadata 决定是否再翻转一次，恢复原始数据。

效果：

- ✅ I/O 翻转下降幅度与传统 DBI 同量级（约**节省 20% 功耗**）；
- ✅ 引入的总存储/带宽开销仅 **0.8%**；

- ✓ 完全不需要改 PHY 或增加 pin。

Solution: Stream Flipping (Pinless DBI)

Challenge. I/O dominates: $100 \text{ TB/s} \times 0.37 \text{ pJ/bit} = 296 \text{ W}$, and conventional DBI needs a **sideband pin** the single-cycle 3D-DRAM link doesn't have.



⇒ **long** per-channel flit streams; each flit compared to the *previous* one.

(a) Without DBI

flit $i-1$:	0 0 ... 0
flit i :	1 1 ... 1
flit $i+1$:	0 0 ... 0

→ every bit toggles twice

(b) Stream flipping: invert flit i

0 0 ... 0	0 meta
0 0 ... 0 (was 1...1)	1
0 0 ... 0	0

→ ≈ 0 toggles + 1 metadata bit / flit

Architectural DBI, no pin:

- **Write:** compare each flit to the previous → invert.
- **Read:** fetch the 1-bit tag; invert if set
- Tag co-located with ECC ⇒ **0.8% overhead, no PHY change** → **20% power savings.**

The full 20% I/O power saving with **no pin** and **no PHY change**.

Slide 24: Solution: Stream Flipping (Pinless DBI) — 与上一 flit 比较取反 + 1bit metadata 嵌 ECC。

12. 挑战 3：105°C 结温下的 DRAM 可靠性（良率 / 保持时间 / 软错误 / 带宽损失）

12.1 问题全景（4 个连锁问题）

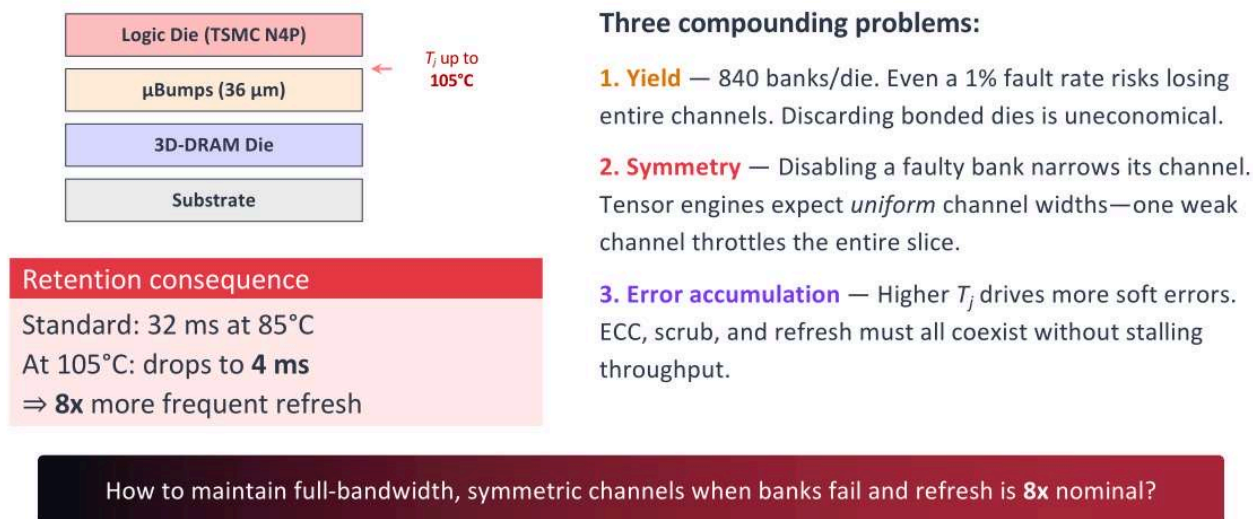
原文定位：Paragraph 21 + Slide 25

当逻辑 Die 工作在 $T_J = 105^\circ\text{C}$ （105 摄氏度结温）时，对下层 DRAM Die 会带来四方面连锁反应：

维度	问题描述	影响
良率 (Yield)	全芯片共 840 个 banks，即使 1% 缺陷率也有 8 个坏 bank	一旦某 channel 的 3 个 banks 被切掉一个 → channel 宽度变窄 → 整张切片 (slice) 被拖累；整颗已键合的 die 丢弃极不经济
通道均匀性	单个弱（窄）channel 会节流 (throttle) 整个 slice 的吞吐	系统性能被最弱的 channel 锁死

维度	问题描述	影响
软错误 (Soft Errors)	温度越高, alpha/中子导致的 DRAM 单粒子翻转越多 → SEU/MBU 风险上升	需要更强的 ECC 与 scrub
数据保持时间 (Retention Time)	85°C 时典型保持时间 32 ms ; 105°C 时骤降到 ~4 ms → 刷新频率要 $\times 8$	若每次刷新都全阵列刷, 有效带宽将损失 1/8 甚至更多

Challenge 3: DRAM Reliability at 105°C



Slide 25: Challenge 3: DRAM Reliability at 105°C — 良率下降 + Retention 32ms→4ms + 软错误上升。

12.2 解法 A: 热感知刷新 + 嵌套 ECC (Reed-Solomon + 通用 ECC)

原文定位: Paragraph 22 + Slide 26

针对刷新风暴 + 软错误的组合拳:

(1) ECC 嵌套机制

在每个子阵列 (subarray) 的最后几列 (last columns) 中交错嵌入 ECC bit 和 DBI bit:

- 底层使用 commodity DRAM 的标准 ECC (通用纠 1 bit);
- 顶层再加一层 **Reed-Solomon 码** (抗多 bit 突发/组错)。

(2) 热感知刷新 (Thermal-Aware Refresh)

Raptor 不会每次刷新都全阵列所有行读一遍。由于:

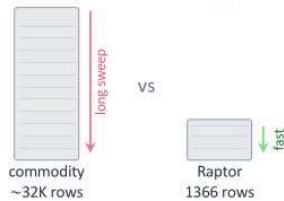
- Stream Blocking、tile 映射等机制已经让"每次真正被激活的 row 数量"减少；
- 实际读出行数比常规 DRAM 访问少 16~32 倍。

因此即使刷新频率 $\times 8$ （从 32ms $\rightarrow$ 4ms），最终被刷到的带宽损失仅约 1.37%，整体仍能维持接近 100 TB/s 的有效带宽（损失 $< 1.4\%$ ）。

Solution: Thermal-Aware Refresh & ECC

Challenge. At 105°C, retention falls 32 ms $\rightarrow$ 4 ms (8x more refresh), and bits still need correcting -- all without losing significant bandwidth.

Deep banking makes refresh cheap



16–32x fewer rows $\Rightarrow$ 4 ms refresh (8x more frequent) costs only 1.37% $\rightarrow$ ~100 TB/s.

Interleaved ECC, co-located with DBI



Last 8 of 124 columns: paired [132,128] Reed–Solomon + DBI bits, interleaved with the subarray.

Read order:



Deep banking turns an 8x refresh penalty into only a $< 1.4\%$ bandwidth loss

Slide 26: Solution: Thermal-Aware Refresh & ECC — ECC/DBI 交错嵌入末列，RS 码 + 通用 ECC 双层纠错。

12.3 解法 B: Bank Chaining (Bank 链式旁路 — 用 spare banks 无缝替换失效 banks)

原文定位: Paragraph 23 + Slide 27

针对"一个坏 bank 变窄 channel、拖累全 slice"的难题，Raptor 设计了**两级物理多路选择器 (Mux)**，实现 Bank Chaining（链式旁路）：

- **第 1 级 Mux**：遇到第 1 个故障 bank 时直接跳过；
- **第 2 级 Mux**：遇到第 2 个故障 bank 时再次跳过。

芯片预留的 **72 个 spare banks** 可以**任意位置回填**芯片任何地方的坏 bank：

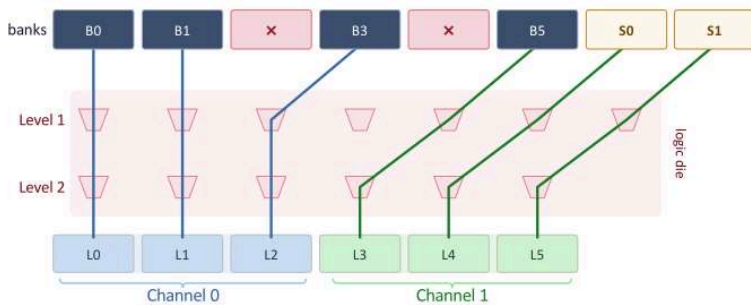
- 通过物理绕线路由把 spare bank 串到对应 channel 的数据流中；
- d-Matrix 宣称额外路由开销"可忽略 (negligible routing cost)"。

最终效果：

- 有/无 bank 失效，各 channel 宽度保持**完全对称（symmetric）**；
- 不存在"弱 channel 节流系统"；
- 键合后即使检测到若干 bank 失效，也可通过熔丝/寄存器重映射，而不用丢弃整颗 die（显著降低量产成本）。

Solution: Bank Chaining

Challenge. The 72 spare banks must cover faults *anywhere* on the die — without making channels asymmetric or adding long edge wires.



*Level 1 skips the first fault (B2); Level 2 skips the second (B4) — **M=2 physical mux levels tolerate 2 faults anywhere**, spares backfill, channels stay symmetric.*

The chiplet's 72 in-line redundant **μbanks** absorb arbitrary faults -- symmetric channels, negligible routing cost.

Slide 27: Solution: Bank Chaining — 两级物理 Mux + 72 个冗余 banks 保证 channel 宽度对称。

13. 结论（一）：相对 HBM4 / NVIDIA Rubin R200 的量级优势

原文定位：Paragraph 24 + Slide 28

d-Matrix 给出一组直接的 Silicon Area & Power 对比（芯片面积 vs 带宽、每 GB/s 功耗）：

指标	Raptor 3D-DRAM	HBM4 系方案（如 NVIDIA Rubin R200）	倍数提升
单位面积带宽	~32.6 GB/s per mm ²	~1.5 GB/s per mm ²	~20×
单位带宽功耗	~2.96 mW per GB/s	~40 mW per GB/s	~13.5×（更...

这是本次演讲最"硬核"的量化结论：把逻辑直接叠到 DRAM die 之上，在面积效率与能效两个维度，相对 HBM 路线都是一个数量级的领先。

Conclusions - 1

Our contribution

3D-DRAM puts bandwidth *and* capacity in one package → three co-designed solutions.

Raptor 3D-DRAM vs. HBM4 & Rubin R200 silicon-area basis

Metric	Raptor 3D-DRAM	HBM4 24 Gb	HBM4 32 Gb	Rubin R200
Capacity (MB/mm²)	11.4	21.9	26.3	21.9
Bandwidth (GB/s/mm²)	32.6	1.67	1.51	1.39
Power per GB/s (mW/GB/s)	2.96	40.0	40.0	40.0

Effective-BW basis (Raptor 83%, Rubin 85% util.). Power per GB/s: lower is better. Rubin R200 = 8× HBM4 24 Gb SoC.

≈20x the bandwidth per mm² and 13.5x better power per GB/s.
The memory wall is a technology problem. 3D-DRAM is the solution.

Slide 28: Conclusions 1 — 单位面积带宽 ~32.6 vs 1.5 GB/s/mm²；单位带宽功耗 ~3 vs 40 mW/GB/s。

14. 结论（二）：~1000 TPS/用户 服务 3T 参数 + 1M 上下文模型

原文定位：Paragraph 25 + Slide 29

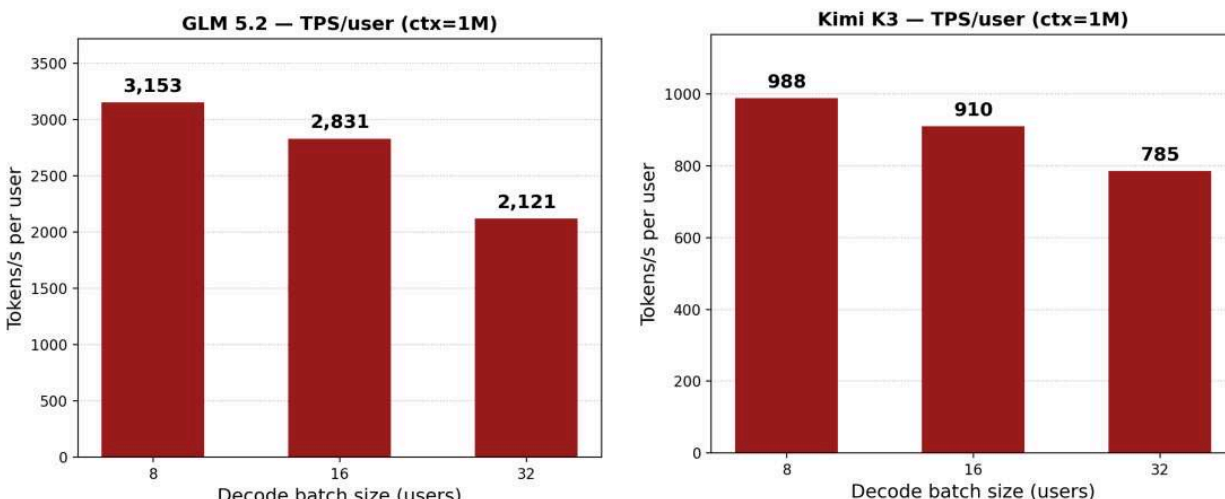
系统级性能宣称：

Raptor 在服务 3 万亿参数级（3T-class）的前沿大模型、1M 上下文长度时，可持续维持 ~1000 tokens per second per user（每用户 ~1000 TPS）的生成速率。

d-Matrix 用这个数据点来支撑其核心论断：

对低延迟 LLM 推理（尤其是边缘/中小集群的 Decode），"适中容量（32GB/卡）+ 超高带宽（100TB/s 级）"的组合是最优解，而非"超大 HBM 容量 + 中等带宽"。

Raptor sustains ~1000 TPS/User for serving 3T class model at 1M context !



Moderate per-card memory paired with high memory bandwidth wins Low Latency Inference

Slide 29: Raptor sustains ~1000 TPS/User for serving 3T class model at 1M context — 中等容量 + 超高带宽 = 低延迟推理王道。

15. 作者点评 (Final Words / 译者工程视角总结)

15.1 原文点评 (ServetheHome 作者)

原文定位: Paragraph 26 + Section Final Words

Raptor 是一次架构级赌博: 它把"容量与带宽打包进同一颗 3D-DRAM 芯片封装"作为 AI 推理的未来方向。其相对 HBM4/Rubin R200 在**带宽密度 (20×)** 与 **单位带宽能效 (13.5×)** 上的量化优势, 构成了强有力的证据链。如果 3D-DRAM 这条路线能真正落地并把内存带宽瓶颈打破, 它将彻底重塑 AI 加速器厂商在"容量 vs 带宽"之间的取舍公式。

一个自然的问题是: "如果这条路这么好, 为什么 NVIDIA 这样的巨头不直接做?" 可以合理推测: NVIDIA 必然有团队评估过逻辑 Die 叠 DRAM Die 的方案; 只是当前主流 HBM 生态已经非常成熟, 换架构成本极高——这正是 d-Matrix 这类创新公司的切入窗口。

15.2 译者的工程视角总结 (补充给 PIM/存算/DRAM 设计读者)

- 路线选择的本质:** Raptor 没有走 DCIM/SRAM-CIM 的"小宏嵌入 SRAM"路线, 也没走 DRAM-PIM 的"bank 旁塞小 MAC"路线, 而是选择了**架构层的第三极——3D 集成 + 近存计算**——把整个"张量引擎 + 互连"叠在 DRAM 头上, 用 I/O 路径的缩短带来一个数量级的能效优势。

2. 与你正在设计的 LP6 PIM PU 的关联：两者在瓶颈定位（Decode 带宽）、DataMapper 2D 分块、Bank/BG 交错映射、可靠性（ECC/刷新/冗余）这些维度高度同构；不同点仅在"PU 与 DRAM 的距离"——你的是 bank 旁的同一 die 级，Raptor 是 3D 堆叠的 mm 级垂直通路。
3. 何时复制这条路线有意义：当系统对单 token 延迟（Decode 阶段）的权重、远超过Prefill 算力密度的追求时；而如果工作负载是训练或大 batch Prefill（算力敏感、HBM 容量敏感），Raptor 相对于传统 HBM 的优势会被削弱。

本笔记内图片索引（全部本地化于 images/ 目录）

图号	Slide	对应本地文件
图 1-1	02	images/dmatrix-raptor-slide-02.jpg
图 2-1	04	images/dmatrix-raptor-slide-04.jpg
图 2-2	05	images/dmatrix-raptor-slide-05.jpg
图 2-3	06	images/dmatrix-raptor-slide-06.jpg
图 3-1	07	images/dmatrix-raptor-slide-07.jpg
图 3-2	08	images/dmatrix-raptor-slide-08.jpg
图 4-1	10	images/dmatrix-raptor-slide-10.jpg
图 4-2	11	images/dmatrix-raptor-slide-11.jpg
图 5-1	12	images/dmatrix-raptor-slide-12.jpg
图 6-1	14	images/dmatrix-raptor-slide-14.jpg
图 7-1	15	images/dmatrix-raptor-slide-15.jpg
图 8-1	17	images/dmatrix-raptor-slide-17.jpg
图 9-1	18	images/dmatrix-raptor-slide-18.jpg
图 9-2	19	images/dmatrix-raptor-slide-19.jpg
图 10-1	20	images/dmatrix-raptor-slide-20.jpg
图 10-2	21	images/dmatrix-raptor-slide-21.jpg
图 10-3	22	images/dmatrix-raptor-slide-22.jpg
图 11-1	23	images/dmatrix-raptor-slide-23.jpg
图 11-2	24	images/dmatrix-raptor-slide-24.jpg
图 12-1	25	images/dmatrix-raptor-slide-25.jpg
图 12-2	26	images/dmatrix-raptor-slide-26.jpg
图 12-3	27	images/dmatrix-raptor-slide-27.jpg

图号	Slide	对应本地文件
图 13-1	28	images/dmatrix-raptor-slide-28.jpg
图 14-1	29	images/dmatrix-raptor-slide-29.jpg

双向关联（直接跳转）

- 同路线竞争 / 标准演进：[Samsung LPDDR5X-PIM at Hot Chips 2026](#)、[papers/2606.00636-lp5x-pim-sim](#)（三星 LPDDR5X-PIM 模拟器）
- 更远期的 M3D-DRAM 查表路线：[PALUTE — M3D DRAM 内 LUT 查询的边缘 LLM 推理加速](#)
- 全景产业路线图：[synthesis/digital-pim-cim-landscape-2026](#)（数字 PIM/CIM 全景 2026）
- 你项目级别的架构规划：[synthesis/cxmt-pim-product-roadmap-2026](#)（CXMT 存算产品路线图）